Understanding User Quality of Experience in Virtual Reality via Multipath QUIC

Dylan Fernandez de Lara
dylan.fernandezdelara@yale.edu

Advisor: Sohee Park
sohee.park@yale.edu

A Senior Project submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science

Department of Computer Science
Yale College
May 1, 2023

Understanding User Quality of Experience in Virtual Reality via Multipath QUIC

Dylan Fernandez de Lara

Abstract

Common network protocols such as TCP dominate the network stack and occupy around 90% of the Internet traffic and bandwidth [1]. Applications such as web browsing, email, and file transfers are supported on networks with the current infrastructure, however with the advent of new applications such as VR streaming that consume higher bandwidth, the quality of these applications cannot be fully supported. Specifically, 360-degree streaming for virtual reality (VR) applications requires high bandwidth connections that common network protocols such as TCP and Multi-path TCP (MPTCP) are not able to efficiently provide [2]. The Quality of Experience (QoE) of VR streaming degrades on existing protocols due to issues such as Head of Line (HoL) blocking [3] and the use of TCP over inherently variable wireless networks. Multi-path QUIC (MPQUIC) is a QUIC extension that attempts to solve these issues by allowing multihoming hosts to aggregate various paths such as LTE or WiFi into a QUIC connection. In particular, we evaluate TCP and QUIC in real environments, along with MPTCP and MPQUIC in emulated environments to gain insight on how each network protocol affects QoE of video streaming. In general, our findings are such: 1) MPQUIC outperforms the other protocols, followed loosely by MPTCP, 2) In the comparisons between QUIC and TCP, there doesn't seem to be a notable QoE performance difference between standard 3G and 4G LTE, so there is no clear benefit to using one network protocol over another, 3) On fast networks, QUIC seems to have an edge over TCP as illustrated by the 51% higher QoE score, and 4) For multi-path networks, MPQUIC outperforms MPTCP and the MP QoE score is 47% better than MPTCP's MP QoE score.

Contents

Chapter 1

Introduction

1.1 Background

Common network protocols such as TCP dominate the network stack and occupy around 90% of the Internet's traffic and bandwidth [1]. Applications such as web browsing, email, and file transfers are supported on networks with the current infrastructure, however with the advent of new applications such as VR streaming that consume higher bandwidth, the quality of these applications cannot be fully supported. The adherence of infrastructure to support emerging technologies has necessitated the development of new protocols such as QUIC [4], [5], Quick UDP Internet Connections, that is able to move the responsibility of the transport protocol from the platform to the application. QUIC utilizes multiplexing datastreams via a single QUIC connection that is maintained between the client and the host via an agreed upon connection ID. This insures that any changes in addressing at lower protocol levels do not cause disruption to the transport of data.

With the increased abundance of multihoming devices such as cellular phones being connected to WiFi and LTE, the combination of these networks via MPTCP [6] have become more prevalent. MPTCP is a protocol that creates a connection split into multiple subflows composed of multiple TCP connections. This allows for the use of multiple paths to increase the bandwidth of the connection, however it still faces issues such as HoL blocking [3] and the constraint of retransmitting data over the same path in order to adhere to middleboxes [7].

As an alternative, QUIC can be used since it offers lower latency and the multiplexity of streams without the HoL blocking issue [5]. Since VR streaming requires significant bandwidth, the use of Multipath network protocols such as MPQUIC [8] and MPTCP could be a viable option to provide a high QoE for the user. As shown

in figure 1.1, multipath networks are able to utilize different network interfaces (i.e. Wi-Fi, ethernet, etc.) in order to increase bandwidth and network speed.

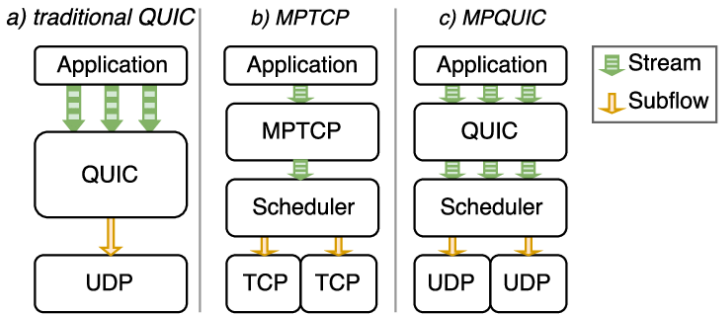

Figure 1.1: Multipath visualization from [9]

Since TCP, QUIC, MPTCP, and MPQUIC have not been evaluated in the context of VR 360-degree streaming, we aim to analyze the performance of these protocols in the given context [10].

1.2 Related Work

Another key area that impacts the bandwidth and latency of a network is the scheduler and its optimizations— making trade-offs in regard to space and time complexity for packet redistribution. The scheduler is responsible for determining when and where packets should be distributed, and specifically choosing which subflows to use for a given stream. There are existing mechanisms that employ a variety of approaches to scheduling such as Round Robin (RR), Stream-Aware (SA) [11], and Reinforcement Learning Based (RL) [12] algorithms that govern scheduling order. Given the development of these protocols, further analysis could be conducted to determine the scheduling mechanism that best improves QoE for streaming and whether or not this differs significantly between various communication protocols.

Chapter 2

Experimental Setup

2.1 Application Setup

The general goal for this experiment was to evaluate the Quality of Experience of each network protocol and quantify a difference between them. The evaluation testbed for this experiment was setup such TCP and QUIC could use the same application in order to ensure fairness. The client used Next.js and Node through a Chromium-based web browser, and the trials were run on macOS Google Chrome Canary using experimental settings and Linux Debian Google Chrome Canary. The client webpage consisted of an embedded YouTube iFrame that connected to YouTube's servers and fetched a video based on id. A 360-degree video was then streamed to the client with playback resolutions set to "auto." This meant that the playback quality was determined through YouTube based on their identified network capabilities.

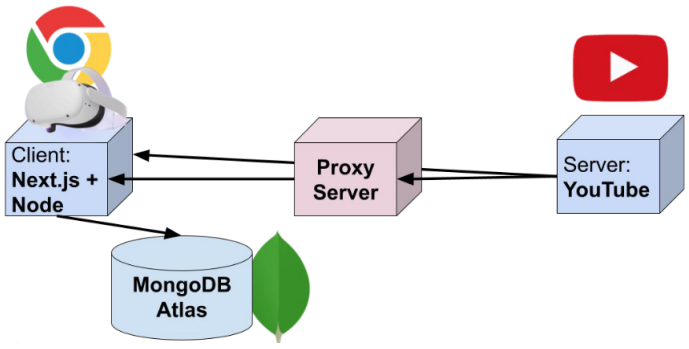

Figure 2.1: Video Streaming Setup

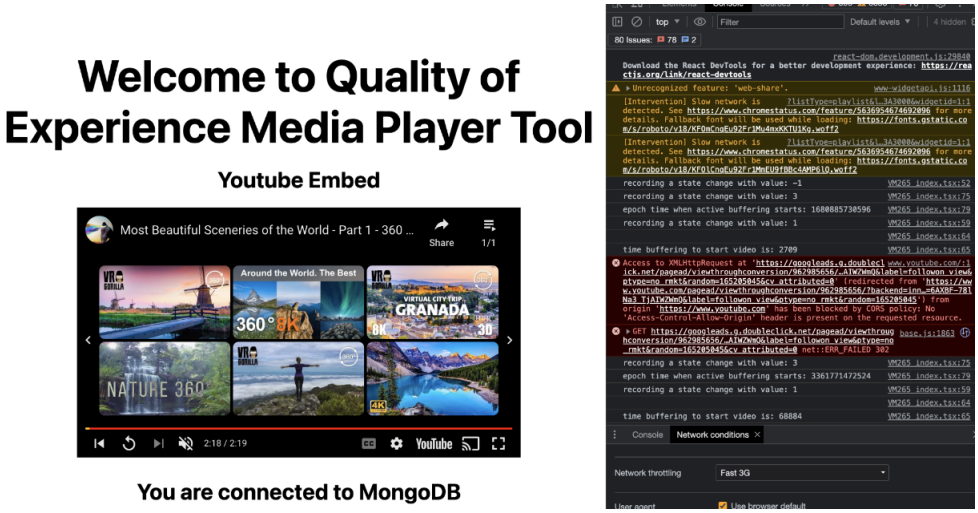

Figure 2.2: Client Page Setup

2.2 TCP

Since all Google services are deployed using QUIC, Google Chrome Canary has experimental flags that allow for toggling QUIC to be on or off. For the TCP setup, the QUIC flag was set to false so native TCP communication was used instead of QUIC.

2.3 QUIC

For the QUIC setup, the QUIC flag was toggled to true to allow for Google Chrome Canary to communicate with YouTube's servers over QUIC. This is accessed at *chrome://flags/#enable-quic* as shown in 2.3 using QUIC version 50 and supporting IETF QUIC draft29.

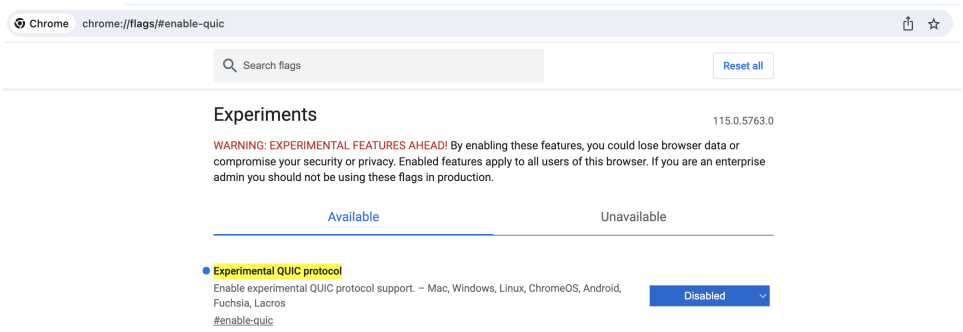

Figure 2.3: QUIC Toggling

2.4 Multipath TCP

The intial Multipath TCP setup required extensive configuration outside of relying on Google Chrome Canary, and the process quickly became cumbersome as YouTube servers don't natively support MPTCP. In order to circumvent many problems, and avoid revisions to the entire evaluation system, a new criteria emerged for the multipath components of this research project. A Mininet environment was configured on Ubuntu 20.04 in Virtual Box and the intial client and server setup was based on figure [13]. By using Mininet and Xterm, a client and server are able to be created, and measured using *iperf*. To setup Mininet, the download instructions were followed to install a Mininet VM on Ubuntu 20.04. *Iperf* was setup with a client and server option, along with the "-i 1" argument specifying that a listening event should be triggered every second.

2.5 Multipath QUIC

The Multipath QUIC setup used a version of MPQUIC called PQUIC [10] that utilizes plugins to enable modularity for the protocol itself. In the experiment, we used a plugin called *multipath_rr_cond*, which is a Round Robin based multipath scheme. Wireshark was configured to track the network traffic between the MPQUIC client and server in order to generalize simulated packet travel for video streaming. It is important to note that the multipath evaluation is limited to testing packet transfer speeds through iperf and Wireshark, and not testing VR streaming capabilities. (This setup was based heavily on the work done by [13]).

Chapter 3

Methodology

3.1 Determining QoE for Single Paths

For VR video streaming, a QOE score was calculated based on metrics recorded from streaming a 360-degree video. In the setup, this meant logging the metrics below to a MongoDB Atlas database and retrieving them later for analysis. To determine Quality of Experience in virtual reality streaming the following key performance indicators were recorded *(time in milliseconds)*:

$$QoE_{score} = a + b + c + d + 2e + 1.5f + g + 0.5h + 0.1i + 0.05j + 0.02k + 0.01l \quad (3.1)$$

where:

a = time spent buffering before playback starts
b = total time spent buffering
c = number of times buffering
d = number of playback quality changes
e = total time played in 2160p
f = total time played in 1440p
g = total time played in 1080p
h = total time played in 720p
i = total time played in 'large'
j = total time played in 'medium'
k = total time played in 'small'
l = total time played in 'mini

For i, j, k, and l, YouTube classifies them using their respective size name, but in actuality they correspond to the 480p, 360p, 240p, and 144p. It is important to note

that this QoE scoring system is novel and does not rely on previously studied work. The intuition behind weighting each variable corresponds with how much I thought it would impact the experience of viewing the stream (i.e. 2160p results in a large multiplier since that is a pleasant viewing experience).

3.2 Experimental Trials

To perform the experiment, trials were setup to evaluate QoE using TCP and QUIC. The same two minute 360-degree video was used and streamed under various network conditions via the Network Link Conditioner (i.e. Low Quality, 3G, 4G LTE, Wi-Fi 802.11ac) as shown in tables 3.1 and 3.2 and figure 3.1. After starting playback of the video, the metrics in equation 3.1 were recorded to compute an overall QoE score for each network protocol.

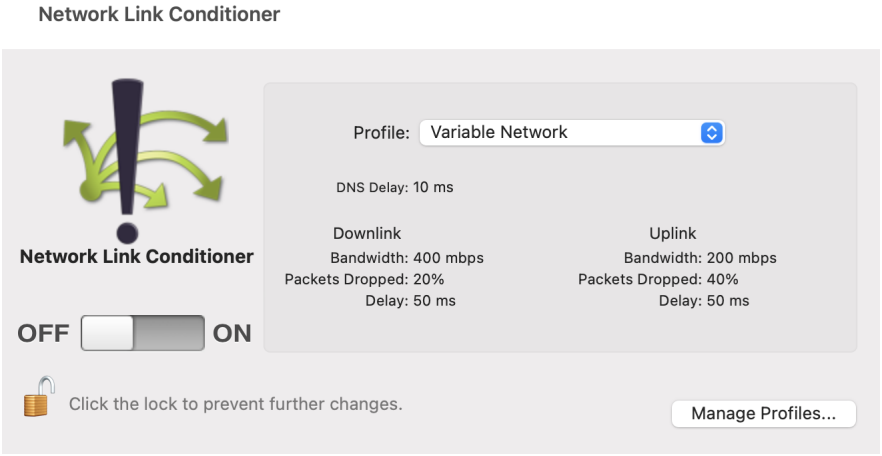

Figure 3.1: Network Link Conditioner

Network Condition	*Bandwidth*	*Delay (ms)*	*Packets Dropped (%)*
3G	780 *Kbps*	100	0
4G LTE	50 *Mbps*	50	0
Variable Network	400 *Mbps*	50	20
Wi-Fi 802.11ac	250 *Mbps*	1	0

Table 3.1: Downlink network condition speeds

Network Condition	*Bandwidth*	*Delay (ms)*	*Packets Dropped (%)*
3G	330 *Kbps*	100	0
4G LTE	10 *Mbps*	65	0
Variable Network	200 *Mbps*	50	40
Wi-Fi 802.11ac	100 *Mbps*	1	0

Table 3.2: Uplink network condition speeds

3.3 Quantifying QoE for Multipath Networks

Classifying the QoE score for MPTCP and MPQUIC looked a bit different than done previously for the single path components. Specifically, network bandwidth and transfer rate were analyzed in order to form a QoE score that could be generalized to 360-degree streaming. (The system challenges will be expanded upon in section 5.2). The equation below is also a novel scoring system and not based on any other literature.

$$MPQoE_{score} = (b + s)/t \tag{3.2}$$

where:

b = network bandwidth
s = bytes transferred
t = time interval

To perform the experiment, the MPTCP setup was configured with *iperf* to analyze packets sent over the network over a duration of ten seconds, along with a comparison to regular TCP. For MPQUIC, the experiment was similar, but instead Wireshark was used to track the number of packets transfered over a duration of time. It is from these metrics that equation 3.2 can be used to create a generalizable QoE score for the multipath components of the experiment.

Chapter 4

Results

4.1 Comparing TCP and QUIC Results

During the trial experiments, the YouTube iFrame did not show significant changes to the playback quality. In the results, this meant that most of the metrics recorded for certain playback qualities remained the same throughout the duration of the experiment with the exception of the *Variable Network* condition.

The data collected shows a minimal difference between the 3G and 4G LTE network conditions highlighting that there is not a notable change in user QoE between TCP and QUIC. For *3G* connections, QUIC underperformed TCP by only 2%, and for *4G* connections, QUIC underperformed TCP by 0.2%. In *Variable Network* conditions QUIC performs 39% better than TCP, and in *Wi-Fi 802.11* conditions QUIC performs 51% better than TCP when comparing QoE score totals. For the experiment run in 4.1, there were five trials for each network condition and each network protocol. The average of all of the values recorded were then used in the QoE scoring system.

It is important to note that the score compiled for the trials is based on a weighting scheme that allows for higher quality streams to improve the QoE score at a rate disproportional to the rate of the lower playback qualities. The total time spent buffering, the total time spent buffering before playback, the number of times buffering, and the number of playback quality changes all contain equal weights, so this can possible affect results as well. This is expanded upon in the *Future Work* section of the report.

To investigate where differences in the QoE score come from, in 4.2 the % playback quality at specific durations were depicted. The intention for creating the graph is the following: Since half of the trials (five streams for each network condition) were

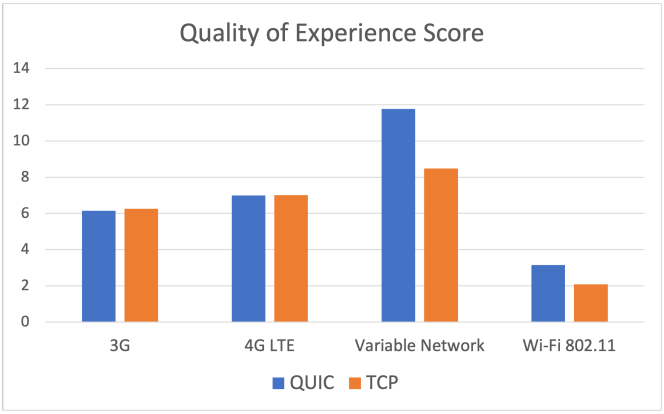

Figure 4.1: Comparing QoE score in various network conditions

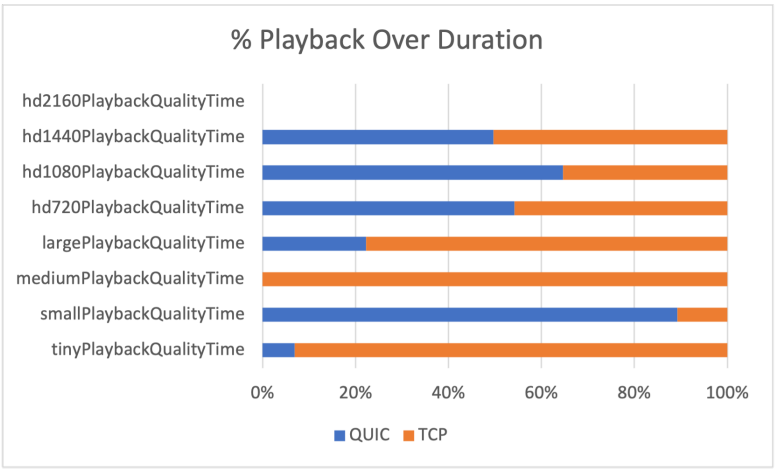

Figure 4.2: Percent playback quality over duration of video

for QUIC and the other half for TCP, then 50% of the total duration of streamed should belong to both halves. By seeing which qualities differed, we could gather insight into which quality each protocol was disproportionately streaming.

It is clear that at the higher streaming qualities (1440p, 1080p, and 720p) the playback duration was about equal between QUIC and TCP, but at lower playback qualities they differed significantly. Neither QUIC or TCP were streamed at the highest resolution which is most likely due to the network speed upperbound.

4.2 Multipath Results

For MPTCP and MPQUIC, the QoE score was calculated using a modified equation as shown in 3.2. To start, there is notable difference in speeds between Multipath TCP and TCP as illustrated in the figures below. The average bandwidth speed in Mbits/sec is 18.1 whereas the same metric is only 9.65 for non-MPTCP enabled communication. This means that with a total interval of 10s, the MP QoE score for MPTCP is 6.13.

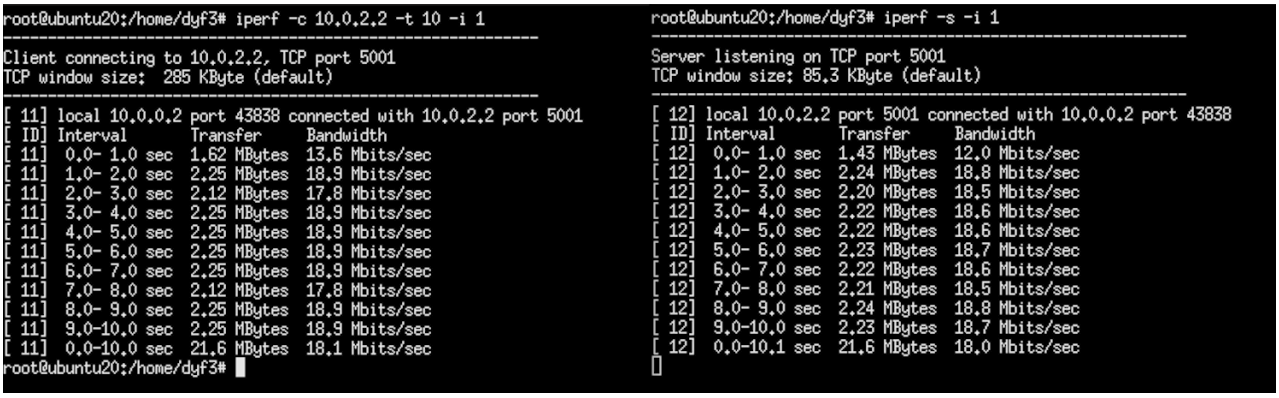

Figure 4.3: MPTCP enabled

For MPQUIC, 100,000 bytes were transferred over a period of 6s with a bandwidth of around 160Mbits/sec on average over the duration of the transfer. This results in an MP QoE score of 26.7; it is clear that there is an advantage with MPQUIC over MPTCP according to the generalizable MP QoE score generated.

```
root@ubuntu20:/home/dyf3# iperf -c 10.0.2.2 -t 10 -i 1
------------------------------------------------------------
Client connecting to 10.0.2.2, TCP port 5001
TCP window size: 86.2 KByte (default)
------------------------------------------------------------
[ 11] local 10.0.0.2 port 43854 connected with 10.0.2.2 port 5001
[ ID] Interval       Transfer     Bandwidth
[ 11]  0.0- 1.0 sec  1.50 MBytes  12.6 Mbits/sec
[ 11]  1.0- 2.0 sec  1.25 MBytes  10.5 Mbits/sec
[ 11]  2.0- 3.0 sec  1.00 MBytes  8.39 Mbits/sec
[ 11]  3.0- 4.0 sec  1.12 MBytes  9.44 Mbits/sec
[ 11]  4.0- 5.0 sec  1.12 MBytes  9.44 Mbits/sec
[ 11]  5.0- 6.0 sec  1.12 MBytes  9.44 Mbits/sec
[ 11]  6.0- 7.0 sec  1.00 MBytes  8.39 Mbits/sec
[ 11]  7.0- 8.0 sec  1.25 MBytes  10.5 Mbits/sec
[ 11]  8.0- 9.0 sec  1.00 MBytes  8.39 Mbits/sec
[ 11]  9.0-10.0 sec  1.12 MBytes  9.44 Mbits/sec
[ 11]  0.0-10.0 sec  11.5 MBytes  9.63 Mbits/sec
root@ubuntu20:/home/dyf3#
```

```
root@ubuntu20:/home/dyf3# iperf -s -i 1
------------------------------------------------------------
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
------------------------------------------------------------
[ 12] local 10.0.2.2 port 5001 connected with 10.0.0.2 port 43854
[ ID] Interval       Transfer     Bandwidth
[ 12]  0.0- 1.0 sec  1.12 MBytes  9.39 Mbits/sec
[ 12]  1.0- 2.0 sec  1.11 MBytes  9.34 Mbits/sec
[ 12]  2.0- 3.0 sec  1.09 MBytes  9.12 Mbits/sec
[ 12]  3.0- 4.0 sec  1.10 MBytes  9.26 Mbits/sec
[ 12]  4.0- 5.0 sec  1.09 MBytes  9.14 Mbits/sec
[ 12]  5.0- 6.0 sec  1.12 MBytes  9.41 Mbits/sec
[ 12]  6.0- 7.0 sec  1.12 MBytes  9.36 Mbits/sec
[ 12]  7.0- 8.0 sec  1.10 MBytes  9.24 Mbits/sec
[ 12]  8.0- 9.0 sec  1.10 MBytes  9.19 Mbits/sec
[ 12]  9.0-10.0 sec  1.10 MBytes  9.24 Mbits/sec
[ 12]  0.0-10.4 sec  11.5 MBytes  9.26 Mbits/sec
```

Figure 4.4: MPTCP disabled

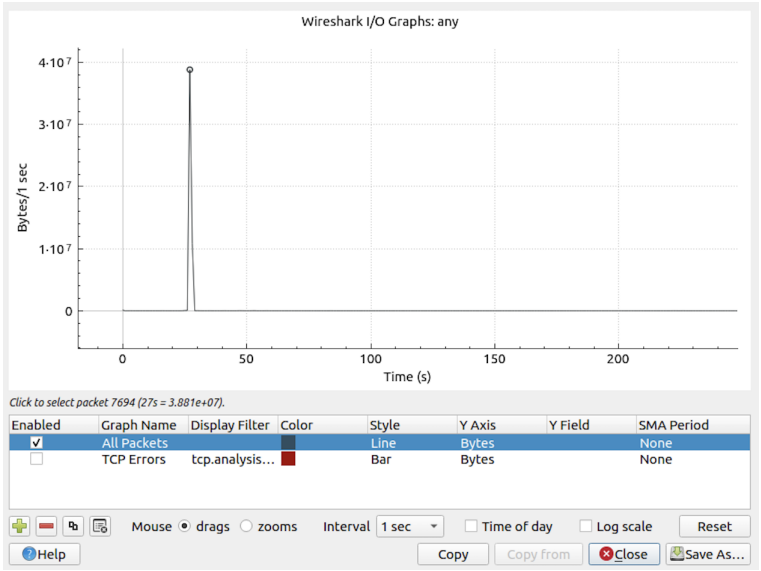

Figure 4.5: MPQUIC bytes/sec over time

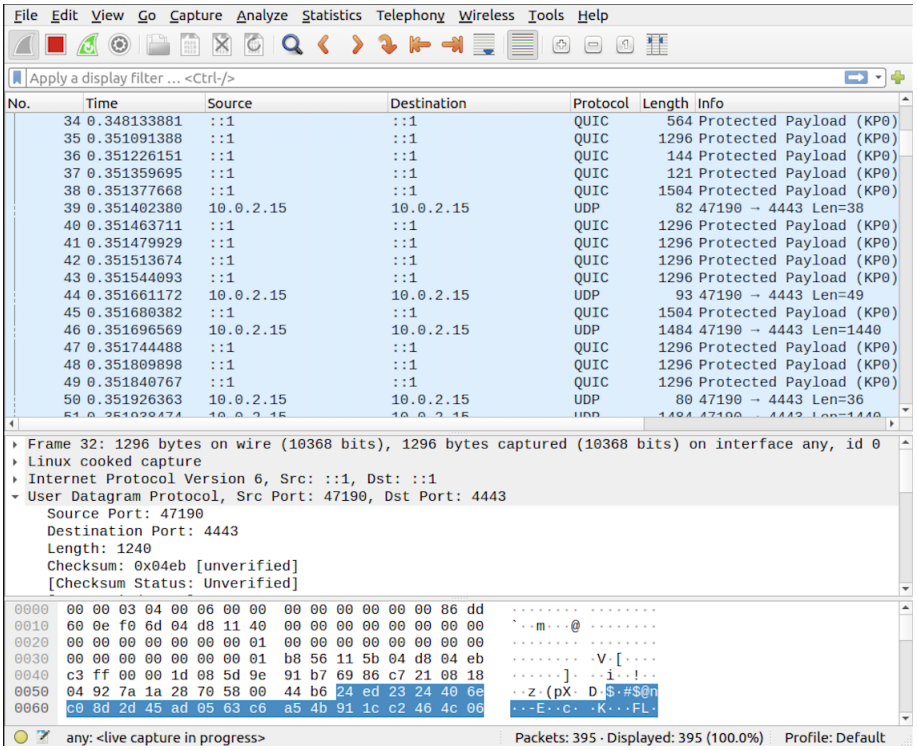

Figure 4.6: MPQUIC Wireshark entries

Chapter 5

Conclusions

5.1 General Findings

In the comparisons between QUIC and TCP, there doesn't seem to be a notable QoE performance difference between standard *3G* and *4G LTE*, so there is no clear benefit to using one network protocol over another. In environments with variable network speeds and high packet drops, there does seem to be a strong QoE when using QUIC over TCP as noted by the 39% higher score. On fast networks, QUIC also seems to have an edge over TCP as illustrated by the 51% higher score.

For multipath networks, MPQUIC outperforms MPTCP and the MP QoE score is 47% better than MPTCP's MP QoE score. Although the multipath findings are not tested in a realistic setting like the single path experiment, this is still a meaningful finding as even in a emulated network setting, MPQUIC performs stronger than MPTCP. With this information, future studies could expand upon these conclusions and test these conditions with real world clients and servers.

With that said, the lack of fully supporting VR streaming over MPTCP and MPQUIC is critical to the project and is a limitation in the multipath component of this report. The experiment provided for multipath combats this by providing an indication of packet performance and estimation for how each protocol would impact QoE. By measuring bandwidth and bytes/sec as shown in equation 3.2, there is a quantitiative measure of multipath protocol performance.

5.2 Areas of Complication

Creating an evaluation testbed such that all network protocols were being compared fairly proved to be a challenge. For TCP and QUIC, Google Chrome Canary was a convenient way to run that portion of the experiment, however creating a real world testbed for MPTCP and MPQUIC was extremely challenging. Most of the issues were in regards to dependencies upgrading and no longer supporting a specific package, or MPQUIC implementations being developed on older versions of Linux and other software. Additionally, creating a Javascript wrapper to run a version of MPQUIC would've been possible, but that wouldn't have actually changed the network routing being done since Chrome is still responsible for that. If given much more time, a project solely could have been detected to developing a Chromium binary capable of running MPQUIC, but for now, the generalizable MP QoE score approach is the next best option.

Chapter 6

Future Work

6.1 Improvements to Evaluation Testbed

In the experiment, there are areas that can be improved upon to create a better evaluation testbed. For the client page as shown in 2.2, there is no way for system to automatically detect the network protocol and update the current protocol variable, and instead it is on the researcher to specify which protocol the experiment is running on. This can be tedious and cumbersome and, if forgotten, could lead to metrics being recorded as the wrong network protocol. Another improvement that could be made is creating an embedded dashboard that displays live metrics as they are being recorded. In this experiment, the data analysis needed to be retrieved from the backend after video playback has ended, so including a dashboard would help the researcher more quickly view how the network protocol is performing.

6.2 Next Steps

There are multiple directions in which this project could be further expanded upon. For one, this wasn't tested extensively on an HMD since the scope of this experiment was contained to network protocols and their effects on QoE. By extending the scope to HMD frame rate, video quality, and frame tears, further evidence could be gathered to quanitfy an all inclusive and more realistic QoE score. Another idea is to obtain Chromium binaries for MPQUIC such that the evaluation testbed remains as fair as possible. Since this isn't fully supported by Google, there is no understanding of the overhead that is caused by testing MPQUIC outside of the direct Google Chrome browser. Most importantly, developing an evaluation test bed outside of

emulated environments will have the greatest impact going forward since that is the most alike to real world environments.

Chapter 7

Acknowledgements

A big thank you to Professor Park for guiding me through this semester-long project. Her willingness to meet and answer questions was always encouraging, and I felt like I learned a lot this semester. Thank you to the Yale CS department for giving me a great four years, and thank you to my family for supporting me all this way! See you at commencement!

Bibliography

[1] Luca Schumann, Trinh Viet Doan, Tanya Shreedhar, et al. "Impact of Evolving Protocols and COVID-19 on Internet Traffic Shares". In: *arXiv preprint arXiv:2201.00142* (2022).

[2] Mohammad Hosseini and Viswanathan Swaminathan. "Adaptive 360 VR video streaming: Divide and conquer". In: *2016 IEEE International Symposium on Multimedia (ISM)*. IEEE. 2016, pp. 107–110.

[3] Quentin De Coninck and Olivier Bonaventure. "Multipath QUIC: Design and Evaluation". In: *Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies*. CoNEXT '17. Incheon, Republic of Korea: Association for Computing Machinery, 2017, pp. 160–166. ISBN: 9781450354226. DOI: 10.1145/3143361.3143370. URL: https://doi.org/10.1145/3143361.3143370.

[4] Adam Langley, Alistair Riddoch, Alyssa Wilk, et al. "The quic transport protocol: Design and internet-scale deployment". In: *Proceedings of the conference of the ACM special interest group on data communication*. 2017, pp. 183–196.

[5] Jana Iyengar, Martin Thomson, et al. "QUIC: A UDP-based multiplexed and secure transport". In: *RFC 9000*. 2021.

[6] Sébastien Barré, Christoph Paasch, and Olivier Bonaventure. "Multipath TCP: from theory to practice". In: *NETWORKING 2011: 10th International IFIP TC 6 Networking Conference, Valencia, Spain, May 9-13, 2011, Proceedings, Part I 10*. Springer. 2011, pp. 444–457.

[7] Sohee Kim Park, Arani Bhattacharya, Mallesham Dasari, et al. "Understanding user perceived video quality using multipath TCP over wireless network". In: *2018 IEEE 39th Sarnoff Symposium*. IEEE. 2018, pp. 1–6.

[8] Yanmei Liu, Yunfei Ma, Christian Huitema, et al. "Multipath extension for QUIC". In: *Proceedings of the IETF Internet Draft, Berlin, Germany* 17 (2021).

[9] Tobias Viernickel, Alexander Froemmgen, Amr Rizk, et al. "Multipath QUIC: A Deployable Multipath Transport Protocol". In: May 2018, pp. 1–7. DOI: 10.1109/ICC.2018.8422951.

[10] Quentin De Coninck, François Michel, Maxime Piraux, et al. "Pluginizing quic". In: *Proceedings of the ACM Special Interest Group on Data Communication*. 2019, pp. 59–74.

[11] Alexander Rabitsch, Per Hurtig, and Anna Brunstrom. "A stream-aware multipath QUIC scheduler for heterogeneous paths". In: *Proceedings of the Workshop on the Evolution, Performance, and Interoperability of QUIC*. 2018, pp. 29–35.

[12] Seunghwa Lee and Joon Yoo. "Reinforcement Learning Based Multipath QUIC Scheduler for Multimedia Streaming". In: *Sensors* 22.17 (2022), p. 6333.

[13] Karthik. *MPStream*. 2020. URL: https://github.com/karthikshetty03/MPStream.